Classification Model Using KNN and Evaluation Metrics

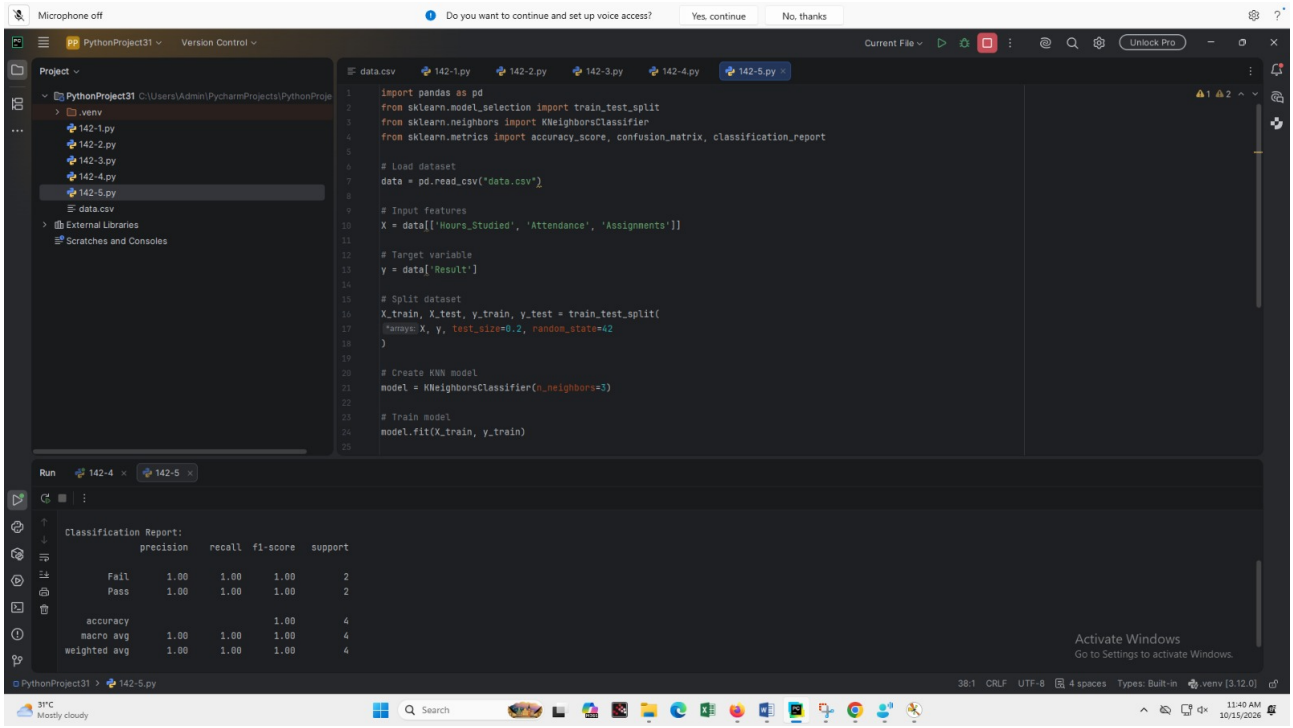
Aim

To write a program that implements a classification model using the K-Nearest Neighbors (KNN) algorithm, trains it on a training set, predicts the class labels for the test set, and evaluates its performance using the confusion matrix, accuracy, precision, recall and F1-score.

Algorithm

1. Start the program.
2. Import the required libraries (pandas, train_test_split from sklearn.model_selection, KNeighborsClassifier from sklearn.neighbors, and accuracy_score, precision_score, recall_score, f1_score, confusion_matrix and classification_report from sklearn.metrics).
3. Load the dataset from the input file.
4. Separate the data into input features (X) and the target variable (y).
5. Split the data into training and testing sets using train_test_split (for example 80% training and 20% testing), with a fixed random_state.
6. Create the KNeighborsClassifier model with a chosen number of neighbors k (for example k = 3).
7. Train the model by fitting it on the training data (X_train, y_train).
8. Predict the class labels for the test set (X_test).
9. For each test record, the model finds its k nearest training records (using Euclidean distance) and assigns the majority class among them.
10. Compare the predicted labels with the actual labels (y_test) and build the confusion matrix (TP, TN, FP, FN).
11. Calculate the evaluation metrics: accuracy, precision, recall and F1-score.
12. Display the confusion matrix and the evaluation metrics.
13. Stop the program.

Program


Microphone off
Do you want to continue and set up voice access?
Yes, continue
No, thanks
PythonProject31
Version Control
Current File
Unlock Pro
Project
PythonProject31 C:\Users\Admin\PycharmProjects\PythonProje
.venv
142-1.py
142-2.py
142-3.py
142-4.py
142-5.py
data.csv
External Libraries
Scratches and Consoles
data.csv
142-1.py
142-2.py
142-3.py
142-4.py
142-5.py
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
Load dataset
data = pd.read_csv("data.csv")
Input features
X = data[['Hours_Studied', 'Attendance', 'Assignments']]
Target variable
y = data['Result']
Split dataset
X_train, X_test, y_train, y_test = train_test_split(
*arrays: X, y, test_size=0.2, random_state=42
)
Create KNN model
model = KNeighborsClassifier(n_neighbors=3)
Train model
model.fit(X_train, y_train)
Run
142-4
142-5
Classification Report:
precision recall f1-score support
Fail 1.00 1.00 1.00 2
Pass 1.00 1.00 1.00 2
accuracy 1.00 4
macro avg 1.00 1.00 1.00 4
weighted avg 1.00 1.00 1.00 4
Activate Windows
Go to Settings to activate Windows.
PythonProject31 > 142-5.py
38:1 CRLF UTF-8 4 spaces Types: Built-in .venv (3.12.0)
31°C
Mostly cloudy
Search
11:40 AM
10/15/2026

Classification Model Using SVM and Evaluation Metrics

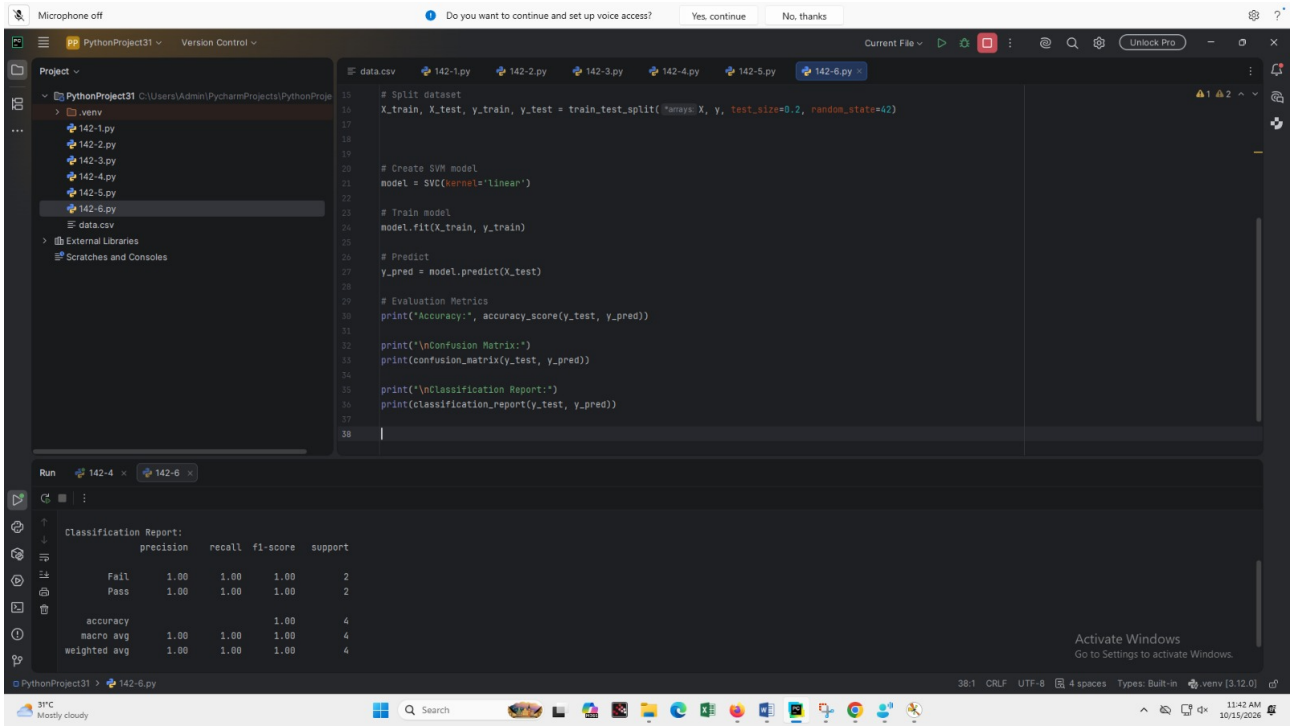
Aim

To write a program that implements a classification model using the Support Vector Machine (SVM) algorithm, trains it on a training set, predicts the class labels for the test set, and evaluates its performance using the confusion matrix, accuracy, precision, recall and F1-score.

Algorithm

1. Start the program.
2. Import the required libraries (pandas, train_test_split from sklearn.model_selection, SVC from sklearn.svm, and accuracy_score, precision_score, recall_score, f1_score, confusion_matrix and classification_report from sklearn.metrics).
3. Load the dataset from the input file.
4. Separate the data into input features (X) and the target variable (y).
5. Split the data into training and testing sets using train_test_split (for example 80% training and 20% testing), with a fixed random_state.
6. Create the SVC model with a chosen kernel (for example linear or rbf) and regularization parameter C.
7. Train the model by fitting it on the training data (X_train, y_train).
8. The model finds the optimal hyperplane that separates the classes with the maximum margin, using the support vectors.
9. Predict the class labels for the test set (X_test).
10. Compare the predicted labels with the actual labels (y_test) and build the confusion matrix (TP, TN, FP, FN).
11. Calculate the evaluation metrics: accuracy, precision, recall and F1-score.
12. Display the confusion matrix and the evaluation metrics.
13. Stop the program.

Program


Microphone off
Do you want to continue and set up voice access?
Yes, continue
No, thanks
PythonProject31
Version Control
Current File
Unlock Pro
Project
PythonProject31 C:\Users\Admin\PycharmProjects\PythonProje
.venv
142-1.py
142-2.py
142-3.py
142-4.py
142-5.py
142-6.py
data.csv
External Libraries
Scratches and Consoles
data.csv
142-1.py
142-2.py
142-3.py
142-4.py
142-5.py
142-6.py
Split dataset
X_train, X_test, y_train, y_test = train_test_split(*arrays: X, y, test_size=0.2, random_state=42)
Create SVM model
model = SVC(kernel='linear')
Train model
model.fit(X_train, y_train)
Predict
y_pred = model.predict(X_test)
Evaluation Metrics
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
Run
142-4
142-6
Classification Report:
precision recall f1-score support
Fail 1.00 1.00 1.00 2
Pass 1.00 1.00 1.00 2
accuracy 1.00 4
macro avg 1.00 1.00 1.00 4
weighted avg 1.00 1.00 1.00 4
Activate Windows
Go to Settings to activate Windows.
PythonProject31 > 142-6.py
38:1 CRLF UTF-8 4 spaces Types: Built-in .venv [3.12.0]
31°C
Mostly cloudy
Search
11:42 AM
10/15/2026